



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

Consigli di Programmazione e Elementi di Base per il Corso

Maura Pintor

maura.pintor@unica.it

In questa lezione

- Introduzione agli strumenti principali
- La shell UNIX
- Struttura della directory
- Esecuzione programmi base
- I file e come sono gestiti dal sistema operativo
- Breve introduzione a Python
 - interpreti e librerie
 - framework
- Gestione del codice
 - versionamento
 - piattaforme collaborative per il codice



Requisiti di base per il corso: premessa

Ogni linguaggio di programmazione ha le sue caratteristiche e i suoi strumenti più usati

In questo corso vedremo degli esempi specifici, ma il focus dev'essere nei **concetti**

- gli strumenti possono cambiare, per esempio possono nascere nuove librerie che sorpassano quelle che vengono usate in questo corso
- il linguaggio di programmazione stesso può cambiare anche all'interno dello stesso web server (oltre che nel tempo), per questo è importante applicare la *modularità*

I comandi specifici e le caratteristiche di ogni strumento non sono importanti da memorizzare se si sa *cosa cercare* e *come deve funzionare*

Nota: Questo corso è strettamente collegato col corso di OOP. Alcuni contenuti saranno ripresi per applicarli alla programmazione web. Non è richiesto seguire entrambi i corsi ma è altamente consigliato.

Requisiti di base per il corso

Durante il corso useremo degli strumenti per implementare un web server e le sue funzionalità

- Un ambiente di sviluppo integrato
- Il terminale
- Python principalmente per il backend
- JavaScript principalmente per il frontend
- SQL e un applicazione per database per le operazioni di lettura e scrittura su disco

Questa lezione ha lo scopo di dare le base su come impostare tutto (e le slide saranno la fonte primaria di informazioni nel caso in seguito serva qualcosa su questi tool)



Integrated Development Environments (IDEs)

Per gestire progetti complessi (non script singoli), è consigliato utilizzare delle applicazioni che integrano più strumenti e consentono automazioni nello sviluppo

- editor di codice sorgente
- completamento automatico, strumenti di analisi del codice (es., per rilevare linee duplicate) e di navigazione rapida (es. salto alla definizione di funzioni o classi)
- debugger e logger
- controlli di versione delle librerie
- compilatori o gestori di interpreti
- ultimamente anche strumenti di AI Generativa per scrittura automatica del codice

I miei IDE suggeriti (ce ne sono tanti altri):

- **Visual Studio Code** <https://code.visualstudio.com>
- **PyCharm e la suite IntelliJ** <https://www.jetbrains.com/pycharm/>



La shell Unix e i principali comandi

Il terminale virtuale (shell)

Un terminale virtuale (o emulatore di terminale), in informatica, è un programma o un servizio del sistema operativo che emula il comportamento di un *terminale testuale*

- dispositivo hardware, che viene usato per inserire dati in ingresso ad un computer e visualizzarli in uscita
- in parole semplici il terminale virtuale è un'interfaccia a riga di comando per interagire con il sistema operativo

Il terminale virtuale è notevolmente utilizzato nelle distribuzioni GNU/Linux per configurarli e inviare comandi

- per convenienza si consiglia di usare Linux / MacOS (Unix) oppure Windows Subsystem for Linux (WSL)
- Alcuni comandi di base possono essere trovati qui <https://terminalcheatsheet.com>



La shell UNIX

La *shell* è un programma, alternativo alle classiche interfacce grafiche (Graphical User Interface, GUI), in cui gli utenti possono inserire dei comandi per il computer

Usare la shell richiede un po' di sforzo in più per imparare i comandi

... ma da molto più controllo sulle operazioni e consente maggiori automazioni

- l'utilizzo è guidato da una serie di istruzioni testuali

Le GUI sono più facili da usare per gli utenti non esperti, in quanto nascondono i comandi

- l'utilizzo è guidato da comandi del mouse e shortcut da tastiera

La grammatica dei comandi shell permette di combinare strumenti esistenti in potenti *pipeline*, e facilita la gestione automatica di più larghi volumi di dati

Perché le GUI non bastano

Nonostante le GUI siano molto facili da imparare a usare, limitandosi al loro utilizzo si rischia di dover fare molto più lavoro

Esempio: immaginiamo di dover creare 100 file di testo in una cartella e chiamarli con numeri incrementali, es. 1.txt, 2.txt, ...

Facendo questo lavoro "a mano", dovrete cliccare tantissime volte con il mouse e interagire anche con la tastiera

- tanto tempo richiesto per completare il task
- possibilità di introdurre errori

Con la shell possiamo automatizzare il processo e farlo diventare incredibilmente più veloce

Fare sempre attenzione ai comandi che si danno alla shell...

Fare sempre attenzione ai comandi che si danno alla shell...

Evitate il famoso

```
rm -rf /
```

Non date assolutamente questo comando!!!

Tra poco saremo in grado di capirlo e sapremo perché si deve evitare

Per ora limitatevi ai comandi usati in questo tutorial finché non capite bene cosa significano



Altri usi utili della shell

La shell è anche molto utile per interagire da remoto con altri computer

Prendere familiarità con la shell ci permette di usare una serie di strumenti e risorse specializzate, inclusi computer ad alte prestazioni, senza bisogno di essere connessi fisicamente

Nel contesto della programmazione web, è molto utile saper usare la shell per copiare il codice del nostro web server e farlo girare *da remoto*



Comandi di base con la shell

Vediamo alcuni comandi di base con la shell

Bash è una shell Unix e anche un linguaggio per istruzioni, impostato come default per Linux e MacOS

Si tratta di un linguaggio *interpretato*, che si contrappone ai linguaggi *compilati*

Maggiori dettagli sui linguaggi interpretati saranno dati in seguito quando parleremo di Python (anch'esso un linguaggio interpretato)



Primi comandi alla shell

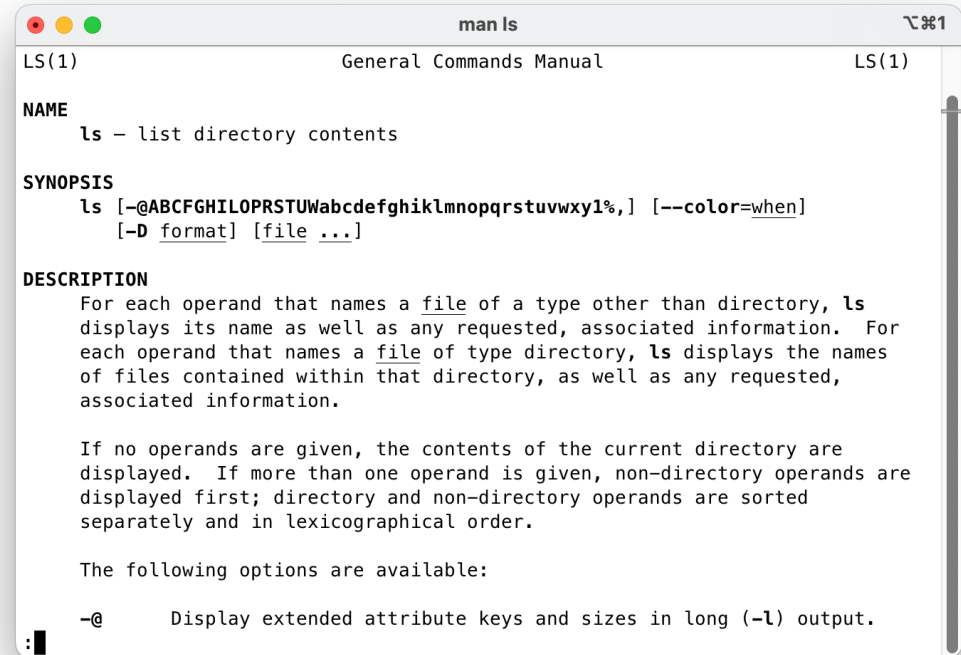
Per iniziare, possiamo chiedere alla shell il manuale di alcuni comandi

I comandi sono programmi pre-esistenti che possiamo richiamare usando il loro nome

Per esempio, il comando `ls` serve a fare la lista dei file in una directory

Possiamo accedere al manuale (anche esso un programma) scrivendo

`man ls`



```
man ls
LS(1)                                General Commands Manual                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [-@ABCFGHILOPRSTUWabcdeghiklmnopqrstuvwxy1%,] [--color=when]
        [-D format] [file ...]

DESCRIPTION
    For each operand that names a file of a type other than directory, ls
    displays its name as well as any requested, associated information.  For
    each operand that names a file of type directory, ls displays the names
    of files contained within that directory, as well as any requested,
    associated information.

    If no operands are given, the contents of the current directory are
    displayed.  If more than one operand is given, non-directory operands are
    displayed first; directory and non-directory operands are sorted
    separately and in lexicographical order.

    The following options are available:

    -@      Display extended attribute keys and sizes in long (-l) output.
```

Eseguire programmi nella shell

Possiamo avviare un programma nella shell scrivendo il suo percorso (path) nel terminale

Tutti i comandi sono eseguiti quando si preme invio nel terminale

Quando un comando è inserito, il sistema cerca il file eseguibile corrispondente nella directory corrente

```
$ ./local_program
```

Se non lo trova, inizia a guardare in altre cartelle in cui di solito si trovano i programmi eseguibili
Alcune sono visibili globalmente (più dettagli nelle prossime slide), quindi è sufficiente scrivere solo il nome dell'eseguibile (per esempio `ls`)

Dove si trovano i programmi della shell?

I programmi della shell (inclusi man, ls), sono chiamati tramite il nome del loro *eseguibile*

La shell definisce una variabile chiamata \$PATH, che include le directory dove si trovano tutti i programmi eseguibili dalla shell

Per vedere la lista di path inclusi nella shell, si può dare il comando:

```
echo $PATH
```

Facendo la lista dei contenuti di uno di questi path, è possibile trovare tutti gli eseguibili a disposizione dell'utente

Per aggiungere un nuovo eseguibile, l'utente può includerlo in una di queste cartelle, oppure aggiungere alla variabile \$PATH il percorso di nuovi eseguibili

I File nel nostro sistema

I file sono raccolte di dati che vengono memorizzati su un dispositivo di archiviazione a lungo termine

Possono contenere vari tipi di informazioni, come testo, immagini, audio o codice di programma
I file possono essere elencati con il comando

```
$ ls -al
```



Esempio di output del comando ls

```
mpintor@atlas: ~  
(base) mpintor@atlas:~$ ls -lah  
total 108K  
drwxr-xr-x 16 mpintor mpintor 4,0K feb 6 2024 .  
drwxr-xr-x 14 root root 4,0K gen 29 11:38 ..  
-rw----- 1 mpintor mpintor 8,7K mag 6 2024 .bash_history  
-rw-r--r-- 1 mpintor mpintor 220 set 12 2022 .bash_logout  
-rw-r--r-- 1 mpintor mpintor 4,2K ott 10 2022 .bashrc  
drwx----- 6 mpintor mpintor 4,0K apr 20 2023 .cache  
drwxrwxr-x 2 mpintor mpintor 4,0K ott 10 2022 .conda  
drwx----- 6 mpintor mpintor 4,0K apr 26 2023 .config  
drwxrwxr-x 3 mpintor mpintor 4,0K feb 20 2023 .dotnet  
drwxr-xr-x 2 root root 4,0K gen 22 2024 external  
drwxrwxr-x 9 mpintor mpintor 4,0K apr 26 2023 fmn_attack_opt  
-rw----- 1 mpintor mpintor 34 feb 6 2024 .lessshst  
drwxr-xr-x 4 mpintor mpintor 4,0K gen 29 2024 .local  
drwxrwxr-x 16 mpintor mpintor 4,0K ott 10 2022 miniconda3  
drwx----- 3 mpintor mpintor 4,0K ott 10 2022 .nv  
-rw-r--r-- 1 mpintor mpintor 807 set 12 2022 .profile  
-rw----- 1 mpintor mpintor 78 apr 26 2023 .python_history  
drwxrwxr-x 3 mpintor mpintor 4,0K ott 10 2022 .pytorch  
drwxrwxr-x 5 mpintor mpintor 4,0K ott 10 2022 pytorch-model-creator  
drwx----- 3 mpintor mpintor 4,0K apr 20 2023 snap  
drwx----- 2 mpintor mpintor 4,0K feb 20 2023 .ssh  
-rw----- 1 mpintor mpintor 1,2K gen 29 2024 .viminfo  
drwxrwxr-x 6 mpintor mpintor 4,0K mag 3 2024 .vscode-server  
-rw-rw-r-- 1 mpintor mpintor 183 mag 3 2024 .wget-hsts  
(base) mpintor@atlas:~$
```

Esempio di output del comando ls

```
mpintor@atlas: ~  
(base) mpintor@atlas:~$ ls -lah  
total 108K  
drwxr-xr-x 1 mpintor mpintor 4,0K feb 6 2024 .  
drwxr-xr-x 1 root root 4,0K gen 29 11:38 ..  
-rw----- mpintor mpintor 8,7K mag 6 2024 .bash_history  
-rw-r--r-- mpintor mpintor 220 set 12 2022 .bash_logout  
-rw-r--r-- mpintor mpintor 4,2K ott 10 2022 .bashrc  
drwx----- mpintor mpintor 4,0K apr 20 2023 .cache  
drwxrwxr-x mpintor mpintor 4,0K ott 10 2022 .conda  
drwx----- mpintor mpintor 4,0K apr 26 2023 .config  
drwxrwxr-x mpintor mpintor 4,0K feb 20 2023 .dotnet  
drwxr-xr-x root root 4,0K gen 22 2024 external  
drwxrwxr-x mpintor mpintor 4,0K apr 26 2023 fmn_attack_opt  
-rw----- mpintor mpintor 34 feb 6 2024 .lessshst  
drwxr-xr-x mpintor mpintor 4,0K gen 29 2024 .local  
drwxrwxr-x 1 mpintor mpintor 4,0K ott 10 2022 miniconda3  
drwx----- mpintor mpintor 4,0K ott 10 2022 .nv  
-rw-r--r-- mpintor mpintor 807 set 12 2022 .profile  
-rw----- mpintor mpintor 78 apr 26 2023 .python_history  
drwxrwxr-x mpintor mpintor 4,0K ott 10 2022 .pytorch  
drwxrwxr-x mpintor mpintor 4,0K ott 10 2022 pytorch-model-creator  
drwx----- mpintor mpintor 4,0K apr 20 2023 snap  
drwx----- mpintor mpintor 4,0K feb 20 2023 .ssh  
-rw----- mpintor mpintor 1,2K gen 29 2024 .viminfo  
drwxrwxr-x mpintor mpintor 4,0K mag 3 2024 .vscode-server  
-rw-rw-r-- mpintor mpintor 183 mag 3 2024 .wget-hsts  
(base)
```

permessi

Esempio di output del comando ls

```
mpintor@atlas: ~  
(base) mpintor@atlas:~$ ls -lah  
total 108K  
drwxr-xr-x 16 mpintor mpintor 4,0K feb 6 2024 .  
drwxr-xr-x 14 root root 4,0K gen 29 11:38 ..  
-rw----- 1 mpintor mpintor 8,7K mag 6 2024 .bash_history  
-rw-r--r-- 1 mpintor mpintor 220 set 12 2022 .bash_logout  
-rw-r--r-- 1 mpintor mpintor 4,2K ott 10 2022 .bashrc  
drwx----- 6 mpintor mpintor 4,0K apr 20 2023 .cache  
drwxrwxr-x 2 mpintor mpintor 4,0K ott 10 2022 .conda  
drwx----- 6 mpintor mpintor 4,0K apr 26 2023 .config  
drwxrwxr-x 3 mpintor mpintor 4,0K feb 20 2023 .dotnet  
drwxr-xr-x 2 root root 4,0K gen 22 2024 external  
drwxrwxr-x 9 mpintor mpintor 4,0K apr 26 2023 fmn_attack_opt  
-rw----- 1 mpintor mpintor 34 feb 6 2024 .lessht  
drwxr-xr-x 4 mpintor mpintor 4,0K gen 29 2024 .local  
drwxrwxr-x 16 mpintor mpintor 4,0K ott 10 2022 miniconda3  
drwx----- 3 mpintor mpintor 4,0K ott 10 2022 .nv  
-rw-r--r-- 1 mpintor mpintor 307 set 12 2022 .profile  
-rw----- 1 mpintor mpintor 78 apr 26 2023 .python_history  
drwxrwxr-x 3 mpintor mpintor 4,0K ott 10 2022 .pytorch  
drwxrwxr-x 5 mpintor mpintor 4,0K ott 10 2022 pytorch-model-creator  
drwx----- 3 mpintor mpintor 4,0K apr 20 2023 snap  
drwx----- 2 mpintor mpintor 4,0K feb 20 2023 .ssh  
-rw----- 1 mpintor mpintor 1,2K gen 29 2024 .viminfo  
drwxrwxr-x 6 mpintor mpintor 4,0K mag 3 2024 .vscode-server  
-rw-rw-r-- 1 mpintor mpintor 183 mag 3 2024 .wget-hsts  
(base) mpintor@atlas:
```

owner e gruppo

Esempio di output del comando ls

```
mpintor@atlas: ~  
(base) mpintor@atlas:~$ ls -lah  
total 108K  
drwxr-xr-x 16 mpintor mpintor 4,0K feb  6 2024 .  
drwxr-xr-x 14 root      root    4,0K gen 29 11:38 ..  
-rw----- 1 mpintor mpintor 8,7K mag  6 2024 .bash_history  
-rw-r--r-- 1 mpintor mpintor 220 set 12 2022 .bash_logout  
-rw-r--r-- 1 mpintor mpintor 4,2K ott 10 2022 .bashrc  
drwx----- 6 mpintor mpintor 4,0K apr 20 2023 .cache  
drwxrwxr-x 2 mpintor mpintor 4,0K ott 10 2022 .conda  
drwx----- 6 mpintor mpintor 4,0K apr 26 2023 .config  
drwxrwxr-x 3 mpintor mpintor 4,0K feb 20 2023 .dotnet  
drwxr-xr-x 2 root      root    4,0K gen 22 2024 external  
drwxrwxr-x 9 mpintor mpintor 4,0K apr 26 2023 fmn_attack_opt  
-rw----- 1 mpintor mpintor 34 feb  6 2024 .lessshst  
drwxr-xr-x 4 mpintor mpintor 4,0K gen 29 2024 .local  
drwxrwxr-x 16 mpintor mpintor 4,0K ott 10 2022 miniconda3  
drwx----- 3 mpintor mpintor 4,0K ott 10 2022 .nv  
-rw-r--r-- 1 mpintor mpintor 807 set 12 2022 .profile  
-rw----- 1 mpintor mpintor 78 apr 26 2023 .python_history  
drwxrwxr-x 3 mpintor mpintor 4,0K ott 10 2022 .pytorch  
drwxrwxr-x 5 mpintor mpintor 4,0K ott 10 2022 pytorch-model-creator  
drwx----- 3 mpintor mpintor 4,0K apr 20 2023 snap  
drwx----- 2 mpintor mpintor 4,0K feb 20 2023 .ssh  
-rw----- 1 mpintor mpintor 1,2K gen 29 2024 .viminfo  
drwxrwxr-x 6 mpintor mpintor 4,0K mag  3 2024 .vscode-server  
-rw-rw-r-- 1 mpintor mpintor 183 mag  3 2024 .wget-hsts  
(base) mpintor@atlas:~$
```

dimensioni

Esempio di output del comando ls

```
mpintor@atlas: ~  
(base) mpintor@atlas:~$ ls -lah  
total 108K  
drwxr-xr-x 16 mpintor mpintor 4,0K feb 6 2024 .  
drwxr-xr-x 14 root root 4,0K gen 29 11:38 ..  
-rw----- 1 mpintor mpintor 8,7K mag 6 2024 .bash_history  
-rw-r--r-- 1 mpintor mpintor 226 set 12 2022 .bash_logout  
-rw-r--r-- 1 mpintor mpintor 4,2K ott 10 2022 .bashrc  
drwx----- 6 mpintor mpintor 4,0K apr 20 2023 .cache  
drwxrwxr-x 2 mpintor mpintor 4,0K ott 10 2022 .conda  
drwx----- 6 mpintor mpintor 4,0K apr 26 2023 .config  
drwxrwxr-x 3 mpintor mpintor 4,0K feb 20 2023 .dotnet  
drwxr-xr-x 2 root root 4,0K gen 22 2024 external  
drwxrwxr-x 9 mpintor mpintor 4,0K apr 26 2023 fnn_attack_opt  
-rw----- 1 mpintor mpintor 34 feb 6 2024 .lessht  
drwxr-xr-x 4 mpintor mpintor 4,0K gen 29 2024 .local  
drwxrwxr-x 16 mpintor mpintor 4,0K ott 10 2022 miniconda3  
drwx----- 3 mpintor mpintor 4,0K ott 10 2022 .nv  
-rw-r--r-- 1 mpintor mpintor 807 set 12 2022 .profile  
-rw----- 1 mpintor mpintor 78 apr 26 2023 .python_history  
drwxrwxr-x 3 mpintor mpintor 4,0K ott 10 2022 .pytorch  
drwxrwxr-x 5 mpintor mpintor 4,0K ott 10 2022 pytorch-model-creator  
drwx----- 3 mpintor mpintor 4,0K apr 20 2023 snap  
drwx----- 2 mpintor mpintor 4,0K feb 20 2023 .ssh  
-rw----- 1 mpintor mpintor 1,2K gen 29 2024 .viminfo  
drwxrwxr-x 6 mpintor mpintor 4,0K mag 3 2024 .vscode-server  
-rw-rw-r-- 1 mpintor mpintor 183 mag 3 2024 .wget-hsts  
(base) mpintor@atlas:~$
```

ultima modifica

Esempio di output del comando ls

```
mpintor@atlas: ~  
(base) mpintor@atlas:~$ ls -lah  
total 108K  
drwxr-xr-x 16 mpintor mpintor 4,0K feb 6 2024 .  
drwxr-xr-x 14 root root 4,0K gen 29 11:38 ..  
-rw----- 1 mpintor mpintor 8,7K mag 6 2024 .bash_history  
-rw-r--r-- 1 mpintor mpintor 220 set 12 2022 .bash_logout  
-rw-r--r-- 1 mpintor mpintor 4,2K ott 10 2022 .bashrc  
drwx----- 6 mpintor mpintor 4,0K apr 20 2023 .cache  
drwxrwxr-x 2 mpintor mpintor 4,0K ott 10 2022 .conda  
drwx----- 6 mpintor mpintor 4,0K apr 26 2023 .config  
drwxrwxr-x 3 mpintor mpintor 4,0K feb 20 2023 .dotnet  
drwxr-xr-x 2 root root 4,0K gen 22 2024 .external  
drwxrwxr-x 9 mpintor mpintor 4,0K apr 26 2023 .fmn_attack_opt  
-rw----- 1 mpintor mpintor 34 feb 6 2024 .lessshst  
drwxr-xr-x 4 mpintor mpintor 4,0K gen 29 2024 .local  
drwxrwxr-x 16 mpintor mpintor 4,0K ott 10 2022 .miniconda3  
drwx----- 3 mpintor mpintor 4,0K ott 10 2022 .nv  
-rw-r--r-- 1 mpintor mpintor 807 set 12 2022 .profile  
-rw----- 1 mpintor mpintor 78 apr 26 2023 .python_history  
drwxrwxr-x 3 mpintor mpintor 4,0K ott 10 2022 .pytorch  
drwxrwxr-x 5 mpintor mpintor 4,0K ott 10 2022 .pytorch-model-creator  
drwx----- 3 mpintor mpintor 4,0K apr 20 2023 .snap  
drwx----- 2 mpintor mpintor 4,0K feb 20 2023 .ssh  
-rw----- 1 mpintor mpintor 1,2K gen 29 2024 .viminfo  
drwxrwxr-x 6 mpintor mpintor 4,0K mag 3 2024 .vscode-server  
-rw-rw-r-- 1 mpintor mpintor 183 mag 3 2024 .wget-hsts  
(base) mpintor@atlas:~$
```

nome file

I permessi dei file

Il primo gruppo di autorizzazioni si applica al **proprietario** del file

Il secondo gruppo di autorizzazioni si applica al **gruppo** di utenti che possiede il file

Il terzo gruppo di autorizzazioni viene generalmente definito "altri"

Ogni carattere dell'espressione indica se un permesso specifico è concesso o meno

- permesso di lettura (r)
- permesso di scrittura (w)
- permesso di esecuzione (x)

```
drwx----- 6 mpintor mpintor 4,0K apr 26 2023 .config
drwxrwxr-x 3 mpintor mpintor 4,0K feb 20 2023 .dotnet
drwxr-xr-x 2 root root 4,0K gen 22 2024 external
drwxrwxr-x 9 mpintor mpintor 4,0K apr 26 2023 fmn_attack_opt
-rw----- 1 mpintor mpintor 34 feb 6 2024 .lessht
drwxr-xr-x 4 mpintor mpintor 4,0K gen 29 2024 .local
drwxrwxr-x 6 mpintor mpintor 4,0K ott 10 2022 miniconda3
drwx----- 3 mpintor mpintor 4,0K ott 10 2022 .nv
-rw-r--r-- 1 mpintor mpintor 807 set 12 2022 .profile
-rw----- 1 mpintor mpintor 78 apr 26 2023 .python_history
drwxrwxr-x 3 mpintor mpintor 4,0K ott 10 2022 .pytorch
drwxrwxr-x 5 mpintor mpintor 4,0K ott 10 2022 pytorch-model-creator
drwx----- 3 mpintor mpintor 4,0K apr 20 2023 snap
```

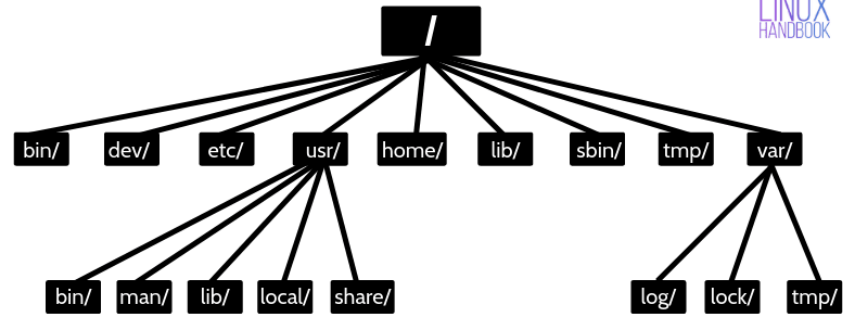
I file

I file contengono altri file, che si diramano dalla radice "/" formando una gerarchia ad albero

- Si trovano in un percorso di cartelle separate da "/", chiamato percorso dei file
- I percorsi che iniziano con "/" sono chiamati percorsi **assoluti**
 - iniziano la ricerca dalla radice del file system
- I percorsi che **NON** iniziano con "/" sono chiamati percorsi **relativi**
 - iniziano la ricerca dalla directory corrente

```
> pwd
/Users/maurapintor
```

Il comando pwd stampa la directory corrente



LINUX
HANDBOOK

Comandi utili

cat <i>file</i>	Concatenate or type out a file
cat <i>file1 file2 ...</i>	Type out a number of files
cd <i>directory1</i>	Change current directory to <i>directory1</i>
cd /usr/bin	Change current directory to /usr/bin
cd	Change back to your home directory
clear	Clear the current screen
cp <i>file1 file2</i>	Copy <i>file1</i> to <i>file2</i>
cp <i>file1 file2 ... dir</i>	Copy a number of files to a directory
ls	List the files in the current directory
ls /usr/bin	List the files in the /usr/bin directory
lpr <i>file1</i>	Print <i>file1</i> out
lpr <i>file1 file2 ...</i>	Print a number of files out
more <i>file</i>	Look at the content of a file with paging, use 'q' to get out
mkdir <i>directory</i>	Create a directory
mv <i>file1 file2</i>	Move <i>file1</i> to <i>file2</i>, like rename.
mv <i>file1 file2 ... dir</i>	Move a number of files into a directory
mv <i>dir1 dir2</i>	Move or rename a directory

Ma quindi, cosa fa il comando `sudo rm -rf /` ?

Vediamo i pezzi che lo compongono:

- **sudo** serve a eseguire un comando come un altro utente (default come root, con permessi massimi)
 - Il comando **rm** serve a rimuovere un file
 - Il parametro **r** serve a rimuovere tutta la gerarchia di file che nasce dalla radice indicata come argomento
 - Il parametro **f** serve a non chiedere conferma prima dell'eliminazione
 - L'argomento **/** è il path radice
- ... Quindi questo comando cancellerà ogni file presente nel sistema!

```
RM(1)                                General Commands Manual                                RM(1)

NAME
    rm, unlink - remove directory entries

SYNOPSIS
    rm [-f | -i] [-dIRrvWx] file ...
    unlink [--] file
```

Il linguaggio Python e il suo funzionamento

Python

Nel corso useremo Python per creare un web server (cosa sia un web server sarà chiaro più avanti)

- non è solitamente il linguaggio più usato per queste cose, ma ci aiuterà per capire le basi senza concentrarci sulle difficoltà di imparare linguaggi nuovi
- è utile a scopo didattico, una volta che si imparano i meccanismi si possono usare altri linguaggi e strumenti più efficienti
 - infatti Python è un linguaggio *interpretato* (vs *compilato*), il che lo rende più inefficiente di altri linguaggi come il C
- consigliata come versione minima la 3.7, in quanto ha introdotto i type hints e librerie utili alla gestione delle chiamate asincrone

Vedremo come installare Python nelle prossime slide



Gestione delle librerie Python

Non scriveremo tutto il codice da zero, ma faremo uso di **librerie software** (collezioni di funzioni e classi scritte da altri e importabili nel nostro codice)

Per Python, il comune strumento per la gestione di pacchetti è pip (<https://pip.pypa.io/en/stable/installation/>), ma ultimamente ce ne sono altri che sono più efficienti e stanno prendendo piede (per il nostro corso andrà benissimo pip)

Tuttavia, è preferibile non avere solo un interprete Python per gestire tutti i progetti che vogliamo scrivere con esso, ma avere un ambiente isolato per ogni applicazione

- Questo consente anche di individuare in modo semplice e pulito tutte le dipendenze necessarie a ogni applicazione
- Nel nostro caso, per esempio, possiamo scrivere il codice nel nostro computer e poi passare la lista di librerie come lista di *requirements*, e ricreare lo stesso ambiente in diverse macchine

Ambienti Virtuali

Con Python, solitamente si può lavorare con versioni di Python e librerie diverse per ogni progetto. Per fare questo, Python consente l'utilizzo di Ambienti Virtuali (Virtual Environments).

- questi non sono altro che cartelle in cui pip scaricherà le librerie
- semplicemente, invece di scaricare tutto in un unico posto, pip separerà tutto in cartelle e importerà queste cartelle solo sugli specifici progetti che lo devono usare

Per semplicità, useremo *Miniconda* (<https://docs.anaconda.com/miniconda/>) per installare Python e poi creeremo gli ambienti virtuali con il sistema integrato di *conda*.

Si può notare che l'installer di conda chiederà di aggiungere il percorso di installazione alla variabile \$PATH.

Questo significa che poi potremo usare conda direttamente scrivendo il nome dell'eseguibile.

Ambienti Virtuali

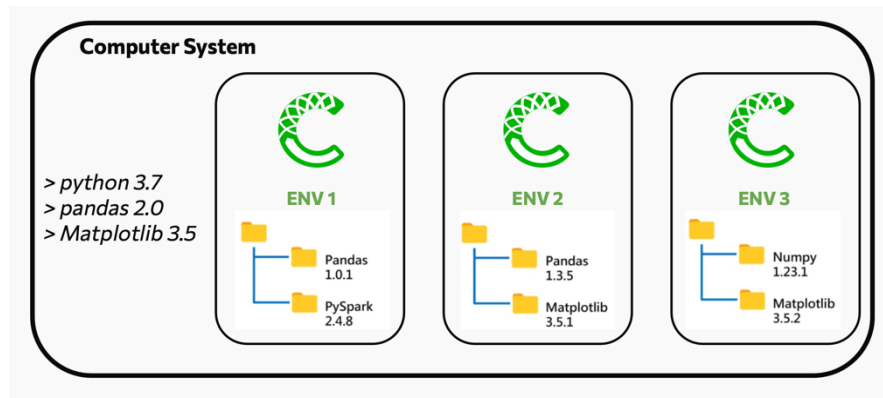
Per creare l'environment

```
conda create -n myenv python=3.9
```

Per attivare l'environment

```
conda activate myenv
```

Ora, ogni volta che si installa un pacchetto con pip, questo lo scaricherà nella cartella dove risiede l'interprete virtuale "myenv"



Ambienti Virtuali

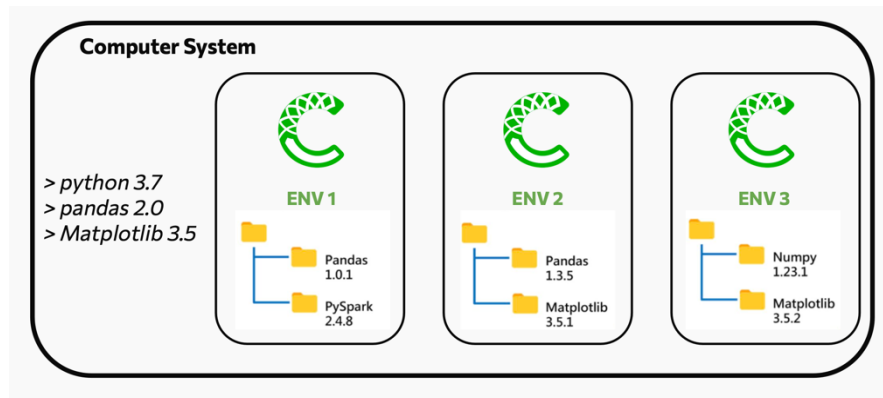
Si può chiedere a conda anche la lista di environment presenti

```
conda env list
```

Questo ci darà il percorso che contiene l'eseguibile python per ogni environment e i relativi pacchetti installati

```
conda env remove -n myenv
```

Quando non sono più necessari, gli environment possono anche essere rimossi



Ambienti Virtuali

Oltre a gestire i pacchetti singoli, pip è in grado anche di gestire scaricamento di pacchetti multipli in file di configurazione

Il metodo più comune è il file di **requirements.txt**, che viene di solito messo in ogni progetto per fare la lista dei pacchetti necessari



Software utili e framework

Un framework è una collezione di software che fornisce delle funzionalità generiche (nel nostro caso di *sviluppo web*) che possono essere cambiate e specializzate per risultare in software specifico per l'applicazione desiderata (per esempio, il nostro server web)

Per esempio, non dovremo scrivere tutto il codice per le richieste HTTP, questo sarà gestito dal framework, mentre noi ci dovremo occupare solo di scrivere le API per le richieste

In questo corso useremo una libreria Python popolare per la sua semplicità di utilizzo e flessibilità: FastAPI (<https://fastapi.tiangolo.com>)

Gestione del codice e versionamento

Importanza del versionamento

Il versionamento del codice è fondamentale per la gestione efficiente dei progetti software, consentendo di tenere traccia delle modifiche e facilitando il lavoro di team distribuiti

- Condivisione degli aggiornamenti
- Cronologia e tracciabilità delle modifiche (e possibilità di annullarle)
- Riproducibilità dei risultati (e dei bug)



```
31     self.file = None
32     self.fingerprints = set()
33     self.logdups = True
34     self.debug = debug
35     self.logger = logging.getLogger(__name__)
36     if path:
37         self.file = open(os.path.join(path, 'requests.json'),
38                           'a')
39         self.file.seek(0)
40         self.fingerprints.update(x.rstrip() for x in self.file)
41
42     @classmethod
43     def from_settings(cls, settings):
44         debug = settings.getbool('SUPERVISOR_DEBUG')
45         return cls(job_dir(settings), debug)
46
47     def request_seen(self, request):
48         fp = self.request_fingerprint(request)
49         if fp in self.fingerprints:
50             return True
51         self.fingerprints.add(fp)
52         if self.file:
53             self.file.write(fp + os.linesep)
54
55     def request_fingerprint(self, request):
56         return request_fingerprint(request)
```

Strumenti per il versionamento: Git

Installazione di Git e configurazione sono abbastanza semplici

- <https://git-scm.com/downloads>

In questo caso avere familiarità con la command line sarà d'aiuto

Ci sono anche strumenti grafici (GUI) per la gestione dei repository

- GUI clients
- Ambienti di sviluppo integrato (IDE)

Vediamo i principali comandi con la command line



Concetti fondamentali

Repository

Un repository Git è un ambiente di lavoro che contiene tutti i file del progetto e la cronologia delle modifiche, sia locali che remoti

Commit

Ogni commit rappresenta uno snapshot della tua applicazione, rendendo possibile il ripristino a una versione precedente in qualsiasi momento

- viene salvato solo "cosa è cambiato", ovvero il "*diff*"

Branching e Merging

Git consente di creare branch per sviluppare funzionalità isolate, mentre il merging integra queste modifiche nel branch principale senza perdere la storia



```
$ git diff
diff --git a/file.txt b/file.txt
index 5d34e82..ba8cc3d 100644
--- a/file.txt
+++ b/file.txt
@@ -1,2 +1,4 @@
 hello
- more text
+ more text!!!
+ a new line
+ yet another line
```

Comandi iniziali di Git

git init

Inizializza un nuovo repository Git locale, suddividendo i file in un'area di staging e rendendoli pronti per il commit

git clone

Permette di copiare un repository esistente da una piattaforma remota, creando una copia locale con tutta la storia e i file

git add

Aggiunge file specifici all'area di staging, preparandoli per la registrazione nel prossimo commit

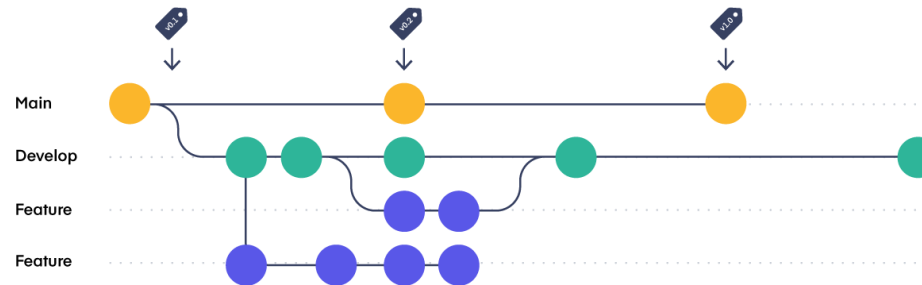
git commit

Registra le modifiche nell'area di staging creando una nuova versione, essenziale per costruire la cronologia del progetto.

Gestione dei branch

In Git abbiamo anche la possibilità di creare dei "rami" (branch) indipendenti di lavoro

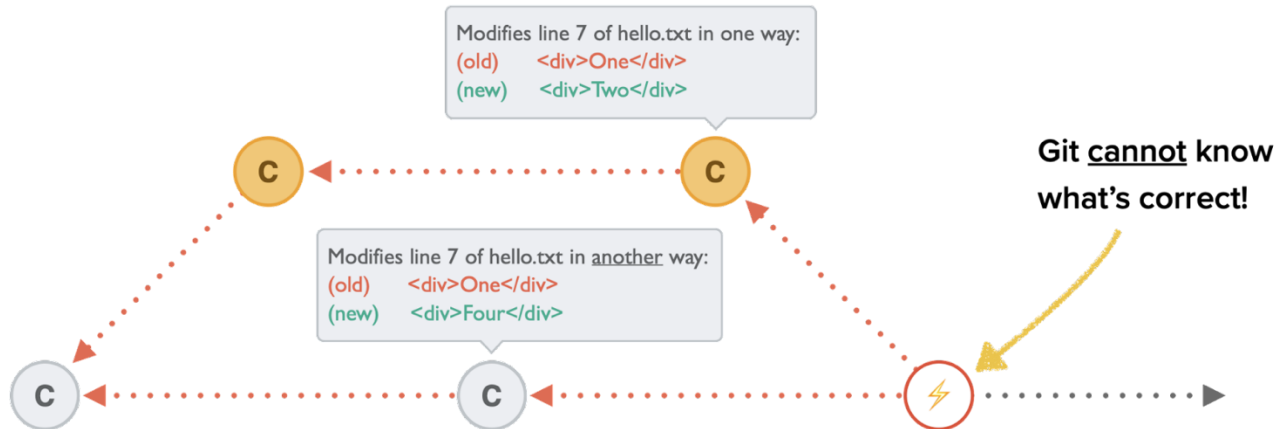
- Un branch principale (*main*) dove risiede la cronologia principale
 - Si possono creare anche branch generati da altri branch
 - Passare da un branch all'altro aggiorna la cartella contenente il repository allo stato attuale di ogni branch (con le modifiche locali)



Gestione dei branch

I branch sono spesso usati per collaborazioni con più sviluppatori

- Usati anche per provare le funzionalità che si aggiungono *prima* di fare modifiche al ramo principale
- L'operazione di unione al branch principale si chiama **merge**
 - Cosa succede se più persone/commit modificano lo stesso file?
 - I **conflitti** devono essere risolti manualmente, richiedendo attenzione ai dettagli e conoscenza del codice



Condivisione del codice e repository online: Github

GitHub (<https://github.com>) è una popolare piattaforma di collaborazione basata su Git

Fornisce hosting per repository e strumenti di collaborazione per progetti di sviluppo

Mentre Git è un **sistema di controllo versione**, GitHub fornisce la capacità di **archiviare repository remoti** e gestire collaborazioni su larga scala

GitHub offre strumenti per la **revisione del codice**, gestione delle **issue** e **integrazioni con CI/CD**, semplificando il flusso di lavoro degli sviluppatori (prossima slide)



Le issue di GitHub

Le issue sono uno strumento integrato in GitHub per segnalare bug, proporre miglioramenti e discutere nuove funzionalità

Permettono di tenere traccia del lavoro da fare in un repository (inclusi i bug fix)

Esempio di repository con gestione delle issue:

<https://github.com/pralab/secml-torch/issues>



La Continuous Integration (CI) e il Continuous Deployment (CD)

Continuous Integration (CI): Automatizza la verifica del codice a ogni modifica, eseguendo test e build in un ambiente controllato

Continuous Deployment/Delivery (CD): Automatizza il rilascio del codice, riducendo il tempo tra sviluppo e produzione

Processo tipico:

- Commit & Push: Il codice viene inviato al repository
- Build: Il sistema compila e prepara il codice
- Test: Esecuzione automatica di test unitari, di integrazione e funzionali
- Deploy: Il codice viene distribuito in ambienti di staging o produzione

Esempio di strumenti per CI/CD: le GitHub Actions

<https://github.com/pralab/secml-torch/actions>



Collegare un repository locale a GitHub

`git remote add`

Questo comando collega il repository locale a un repository remoto su GitHub, consentendo la sincronizzazione delle modifiche

- Bisogna passare l'URL del repository su GitHub (*remote*) per connetterlo

Push e Pull

Il comando 'push' invia le modifiche locali a GitHub, mentre 'pull' recupera le modifiche remoti garantendo coerenza tra centralizzato e locale

Sincronizzazione dei cambiamenti (Sync)

La sincronizzazione regolare tra locale e remoto è vitale per prevenire conflitti e assicurare che l'ultima versione del codice sia sempre disponibile (ed evitare **conflitti**)



Collaborazione su GitHub

Forking

Il forking permette di creare una copia del repository originale, consentendo modifiche personali senza influenzare l'originale (su cui spesso non si hanno i permessi di scrittura)

- Usato per sviluppare una feature indipendente in un proprio account GitHub e poi richiedere l'aggiunta in un repository principale

Pull Requests

Le pull request consentono agli sviluppatori di proporre modifiche al progetto originale, facilitando la revisione da parte degli altri collaboratori

<https://github.com/pralab/secml-torch/pulls>



Best practices per Git e GitHub

Commit frequenti

Effettuare commit frequenti per ogni modifica incrementale contribuisce a mantenere una cronologia chiara e gestibile

Messaggi di commit chiari

Scrivere messaggi di commit informativi e descrittivi è fondamentale per facilitare la comprensione delle modifiche apportate

Documentazione del codice

Fornire documentazione chiara e dettagliata migliora notevolmente la possibilità di collaborare efficacemente e riduce il tempo necessario per comprendere il codice